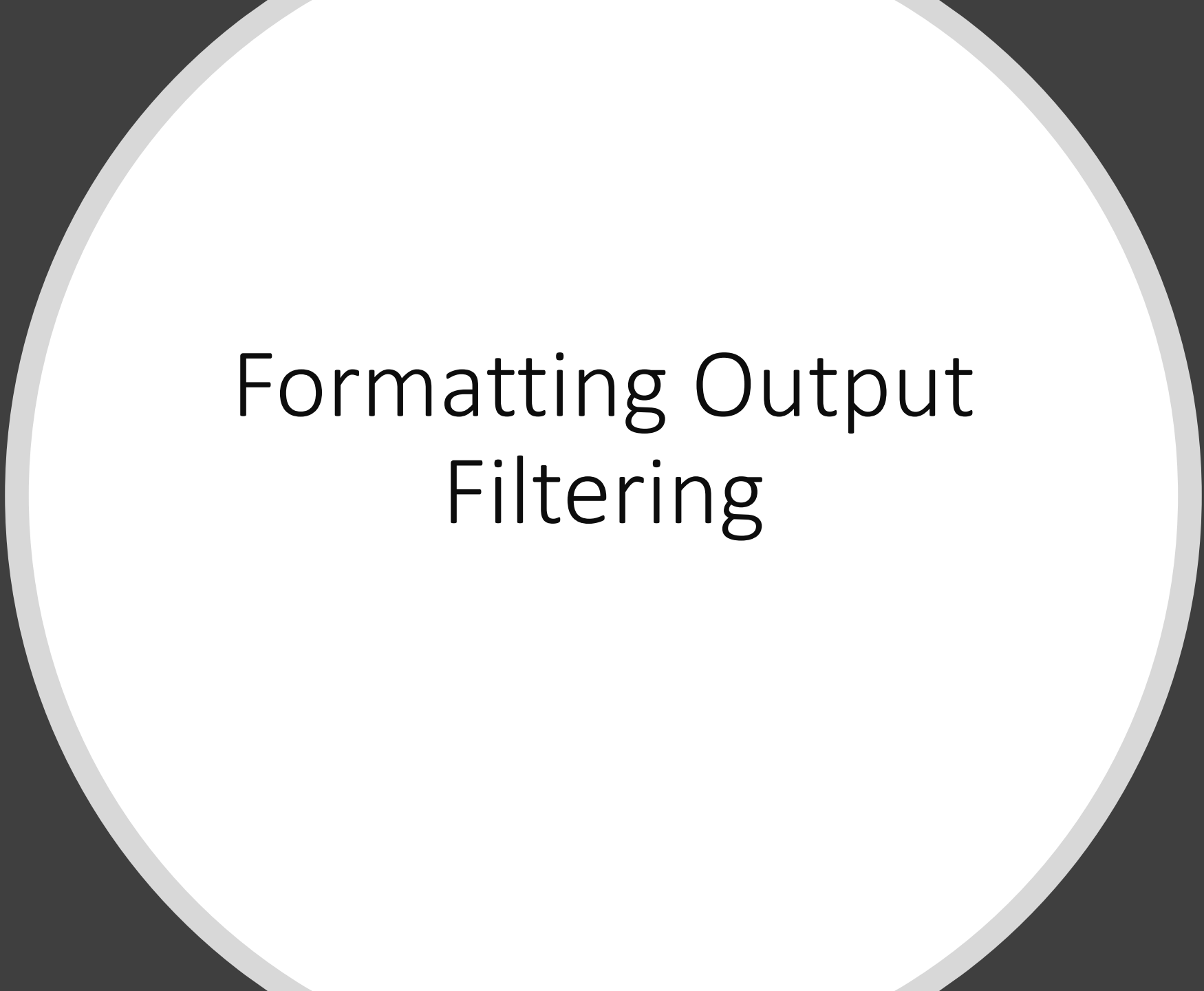




Formatting Output Filtering

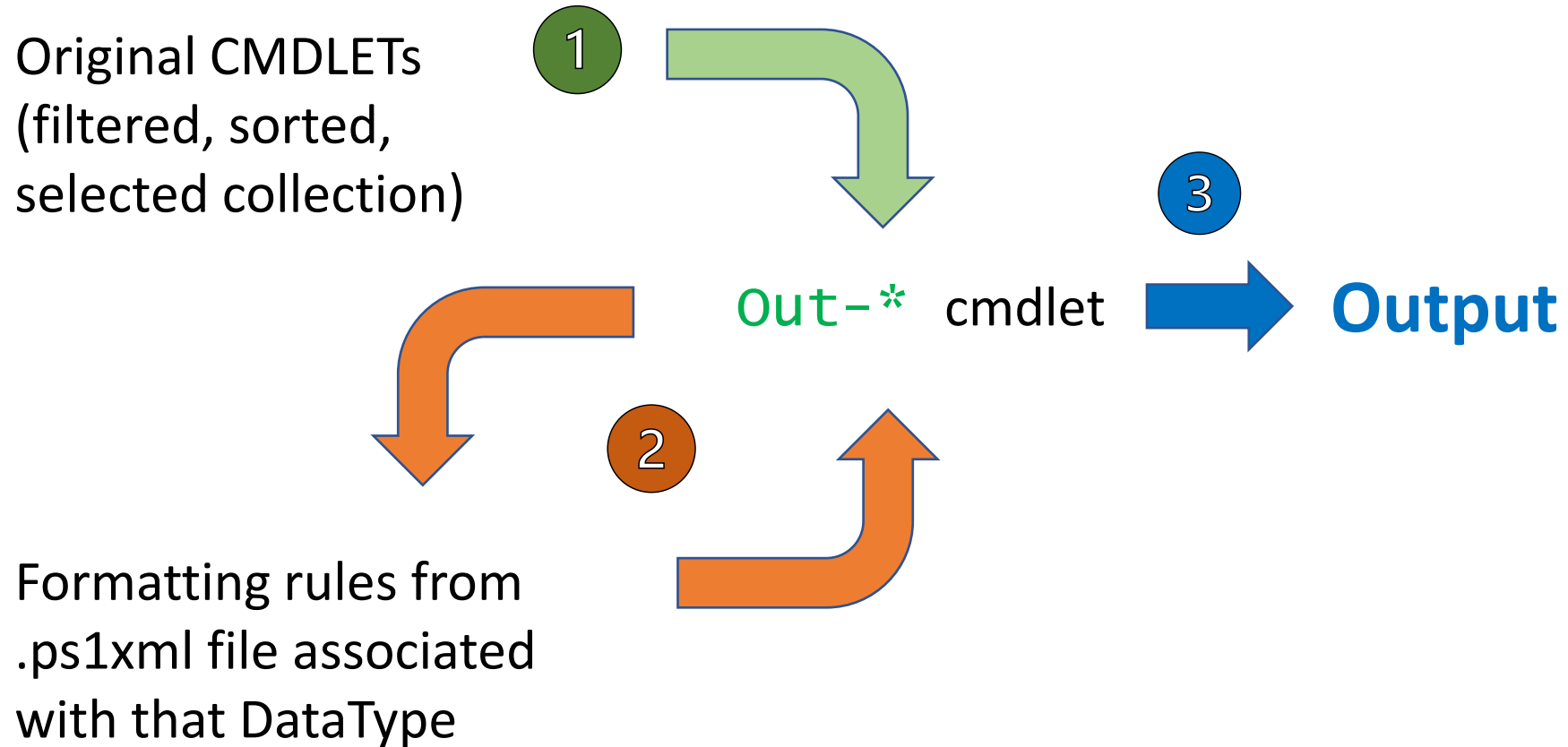
Review of Concepts

- Powershell produces **collections** as output.
- Those objects have **properties** (many of them; more than are displayed by default). We can find the types of properties by using **Get-Member**.
- If you run a command – and do nothing else – Powershell will use a default format to display the output. This default format is **built-in to PS**.
- We can change the default output, however.
 - We can use **Sort-object** to change the ordering of the rows in the collection.
 - We can use **Select-Object** to select the properties (columns) we want displayed in our table.
 - We can use **Format-** commands
 - We can use an **Out-** command to change how the object is output.
- When we do a **Select-Object**, it creates a custom table, that is kind of ugly, because we have told PS not to use the default output formatting.

How Powershell formats Output (by Default)

- Every command has a predefined set of rules to format by.
- The formatting rules are stored with the command's modules in a `.format.ps1xml` files. For example: formatting for `*-Process` commands are store in `DotNetTypes.format.ps1xml`
- Every command does an `Out-Default` after being executed
- If no other `Out-*` command is entered, `Out-Host` will be used (the 'default out' is the host).
 - If the data in the pipeline is a string → Directly to `Out-Host`
 - If the data is not a string → determine if there is a view for this object.
- The `Out-*` command passes the data to the formatting process, applies the formatting view specified for the module (which follows *three rules*), and the data is passed back to the `Out-*` command.

Formatting Flow



Rules of Formatting

(ordered steps used during the formatting process)

- **Step 1:** Attempt to format using **Objects predefined DataType view**
 - If the object is listed in a **.format.ps1xml** formatting file, it will be formatted based on the predefined format rules in this file (like **DotNetTypes.format.ps1xml** for ***-Process** commands).
- **Step 2:** Use **types.ps1xml** for custom formatting
 - If there is a **.types.ps1xml** file for this command specifically, use it.
 - If the object isn't listed in a **.type.ps1xml**, use **all** of the object's properties and make an ugly table.
- **Step 3:** Choose to display in **list**, or **table** format.
 - If the output is displaying **4 or fewer** properties, display a table.
 - If the output is displaying **5 or more** properties, use a list.

Breaking the rules

- We can override the formatting on any table using custom formatting cmdlets.
- There are four formatting commands we can use to change the formatting of our output.
 - **Format-Table**
 - **Format-List**
 - **Format-Wide**
 - **Out-GridView**



Format-Table (FT)

Useful Parameters

- **-autoSize**
 - Normally, PS tries to fit the console windows on the screen. It will fill the screen with the largest table possible (except if there is a predefined view).
 - If there are only 3 columns PS will try to stretch them across the screen.
 - **-autoSize** make PS fit the columns to their contents instead of the console width.
- **-property**
 - You can select the properties to display in the table. If you do, it will list the *real* name of the property instead of the column header stored in the formatting file.
- **-groupBy**
 - Separates the returned objects into different tables (one after each other) based on the property you've grouped by
- **-wrap**
 - Wraps the contents of a value to the width of the column. Adds a '...' after the content if it would be larger than the column.

Format-List (FL)

- Sometimes there is too much information to display for a Table to be practical.
- **FL** can be useful for displaying available properties for an object (like **GM**), but it also shows the *values* of those properties.

Useful parameter

- **-Property** (**-property ***)
 - Selects the properties to be shown in the list.
- **-groupBy**
 - Separates the returned objects into different lists (one after each other) based on the property you've grouped by

Format-Wide (FW)

- **FW** displays “wide lists”.
 - Can only display one property at a time in a Wide format
 - Uses the Name property by default
 - Defaults to two columns.
- Useful Parameter
- **-property**
 - Selects the property to display (default is Name)
 - Doesn't accept wildcards
- **-col**
 - Determine how many columns to display (default 2)

Controlling Output

- **Out-Default** is on the end of every command. It applies the default formatting rules found in PS and then sends the output to an **Out-Host**
- **Out-Printer** sends output to a print device.
- **Out-File** sends output to a text file.
- **Out-GridView** (not really a formatting command) sends the output of a command to table Window.
 - It completely bypasses formatting.
 - It doesn't attempt to format the output at all, but the resulting table can be ordered or formatted like any other table in Windows.
 - The resulting table can be filtered using the GUI it provides.

Tips for using **Format**– Commands

- Always use them *last* in a statement (*format right*)
 - Remember that the **Out**– commands *consume* the formatting instructions from the previous command.
 - There is always an **Out-Default** at the end of every command, unless specified otherwise.
- Example:
 - If you're Going to use **Out-GridView**, format the table before piping to it.
`Gps | Format-Table -Property Name,ID,CPU | Out-GridView`
- You can't put more than one thing in the pipeline if your going to format it.
 - Consider **Get-Process; Get-ChildItem**
 - We can send two commands to the shell (not pipped but running serially) using an “;”

Points to remember

- **Filtering** should be done first (either `-filter` parameter or `where-object`). *Filter left.*
- `Sort-Object` should be used before `Select-Object`.
- `Select-Object` should be used after `Sort-Object` but before `Format-` commands, as it can change the DataType in the pipeline.
- Always `Format-` to the right of your command (right before your standard out is processed). *Format Right.*
- Don't format multiple data types (more than one command being run).

The order of a statement

Filter → Sort → Select → Format → Output

Filtering output.

When there's too much data...

- Filtering data, allows you to parse-out the information you need from the output of a command.
- There are two “models” for narrowing the results of a command.
- Filtering by instructing the cmdlet itself to return a set of objects
 - Use the commands parameters to filter its output (like the `-Property` parameter of `Get-Process`). (*Early filtering*)
`get-process -name explorer`
- OR Iterate the command to another command to filter in (or out) the set of objects you want to work with.
 - Commonly done using `where-object`.

Early Filtering

- This method should be used ***as much as possible***.
- And example of early filtering is simply telling a command what you want to see, and what you don't.
 - `Get-Service -Name A*,D*`
 - `GCI -path *.log,*.csv`
- While, in the above example, we are filtering what we want, it's a *simple* filtering and cannot filter in more complicated ways (like returning only the running services).

Filtering to the left

- This is an idiom that means we should “try to filter as close to the command as possible” (or at the leftmost position of the command).
 - `Get-ADuser -filter *`
 - `Get-ADuser -filter 'Name -like "Sean Mattis"'`
 - The earlier you filter out unwanted objects, the less work the CLI must do to parse your commands.
- Because many commands use `-filter` (in different ways) the ability to filter output varies in both method and functionality.
 - Just because you can filter a command one way doesn't mean it will work with a different command.
 - Example: `Get-Service -filter` doesn't work.
- Using “filter to the left” requires that you be familiar with *many* commands (one of the **problems** with simple filtering).

Comparison Operators and `where-Object` (`where`)

- The command we use to filter the output of another command is `where-Object`.
- With `where-Object`, we often use `Comparison Operators`, to instruct `where` on how to filter.

Comparison Operators

- Comparison Operators take two values and **test their relationship to each other**.
 - Are there two things *equal*?
 - Is things *one greater than* or *less than* thing *two*?
 - Does this thing *match* that thing.
- The *result* of comparing two objects is always **Boolean** (**\$true/\$false**)
...always.
- Either the thing *is* as we've specified, or it *isn't* like the things we've specified.

Type	Operator	Comparison test

Equality	-eq	equals
	-ne	not equals
	-gt	greater than
	-ge	greater than or equal
	-lt	less than
	-le	less than or equal
Matching	-like	string matches wildcard pattern
	-notlike	string does not match wildcard pattern
	-match	string matches regex pattern
	-notmatch	string does not match regex pattern
Replacement	-replace	replaces strings matching a regex pattern
Containment	-contains	collection contains a value
	-notcontains	collection does not contain a value
	-in	value is in a collection
	-notin	value is not in a collection
Type	-is	both objects are the same type
	-isnot	the objects are not the same type

Examples of Comparison Operator Expressions

(numerical/strings)

- `-eq`
 - `50 -eq 34 = $False`
 - `50 -eq 50 = $true`
 - `50 -eq 51 = $false`
 - `-ne`
 - `50 -ne 51 = $true`
 - `50 -ne 52 = $true`
 - `50 -ne 50 = $false`
 - `'Egg' -eq 'egg' = True`
 - `'Egg' -ceq 'Egg' = True`
 - `(5 -gt 10) -and (10 -gt 100) = $false`
 - `(5 -gt 10) -or (10 -lt 100) = $true`
 - `-Not` can be used to reverse the logic of the expression.
 - `$_Responding -eq $False = True!`
 - `-not $_Responding = True!`
 - `! $_Responding = True!`
- (These would accomplish the same thing)
- `get-Process | Where-Object {-not $_Responding}`

Examples of Comparison Operator Expressions

(literal strings)

- **-like**

- Accepts wildcards
- `"Hello" -like "*ll*" = $true`

- **-notlike**

- Accepts wildcards
- `"Hello" -notlike "*ll*" = $false`

- **-match**

- Compares text and regex (more later)

- **-notmatch**

- reverses the logic of match (more later)

Other Comparison Operators

- Some **older commands** will use more traditional comparators.
- If the *-comparator* is not an option, try:
 - **=** equals
 - **<>** does not equal
 - **<=** less than or equal to
 - **>=** greater than or equal to
 - **>** greater than
 - **<** less than

Notes about command inconsistencies

- The designers of the commands choose which comparison operators will be used.
- Getting used to which comparison operators work with which commands is a matter of experience.
- *Read the help files.*

Filtering from the pipeline (interpretation)

- Example

```
Get-Service | where -filterScript {$_.Status -eq  
'Running'}
```

- The **-filter** parameter is positional (0), so we usually don't type it.

```
Get-Service | where {$_.Status -eq 'Running'}
```

- Filter out any process ID above 1000?
- Filter out any process ID above 1000 and below 500?

Filtering from the pipeline (interpretation)

Example:

```
Get-Service | where {$_.Status -eq 'Running'}
```

1. Each object passed from `Get-Service` (each row) is checked against `where's $_.<property>` placeholder. The thing *after* the period is property being checked against. In this case, the property is `Status` and it has two states: `Running` and `Stopped`.
2. `where` checks the first row; checks to see if the service object in that row has a status of `Running` (the property we filtered for is `$true`).
3. If the property is "`Running`" → Include the object in the final table output → Send the object to `Default-Out` and start the formatting process.
4. Continue the process with all objects in the original output that meet this criteria. Filter out any that don't.

Things to remember

- Filter to the left (as much as possible) to simplify what is being passed in the pipe.
- If possible, *don't*, use a second command (**where-Object**) to filter (providing the first command supports filtering of any kind).
- Also, filter as close to the *source* of the data as well. If you are running a remote PS command, filter *on the system* that is generating the data (more about this when we use remote PS commands).
- Depending on what's in the pipeline, **where** may not do what is expected. Make sure the output going into the pipe is the output you expect (consider **Select-Object**).

Things to remember

Consider the example from the book:

```
Get-Process -Name (Get-Content C:\Names.txt | Where-Object -filter {$_ -notlike '*daemon'}) |  
Where-Object -filter {$_.WorkingSet -gt 128KB}
```

1. Run **Get-Content** against **C:\names.txt**
2. Use **where-Object** to filter *in* anything *string* that *doesn't* end in **'*daemon'**. This will give us an array of strings that don't end in 'daemon'. This will be used as input for **Get-Content**.
3. The array parsed out by **Get-Content** will be sent to the **-Name** parameter of **Get-Process**. **Get-Process** will display all processes that do not end in **'daemon'**.
4. The collection from **Get-Process** is passed through the pipe to the next where command. The collection contains a table of all processes, except the processes that end in **'daemon'**.
5. The Second **where-Object** command will filter the collection in the pipeline to display any process, who's name does not end in daemon, and has a **working set** (memory usage) that is Greater than **128 Kilobytes**.

The order of the statement

- Filter:
 - To left most position
 - filter early.
 - Filter close to the data source
- Sort the data before selecting it.
- Select the objects to make your table.
 - This changes our object type.
- Apply output formatting
- Determine Output method.
- **Book Example**: How would find the most CPU-intensive process, while excluding the Powershell Process from the results; then sum the amount of Memory the top 10 processes are using?

Consider this example

Challenge:

- Show me all the files in the C:\work directory, including files in the subfolders
- do not show files that have a size of 0 bytes
- do not show folders
- sort them from largest to smallest
- show only the name and size of the files on the screen in a table.